# AP CSA Keywords & Modifiers Reference

Access, static, final, void, this, super, new, null, extends - and their traps

Every keyword the exam actually uses, what it does, and the mistake each one invites. Grouped by job: **access**, **sharing**, **protection**, **object machinery**, **control flow**.

## 1. Access Modifiers: Who Can See It

| Modifier | Visible to | Where you use it |
|---|---|---|
| `public` | Every class in the program | Classes, constructors, methods the driver calls |
| `private` | The declaring class only | Instance variables and helper methods |
| `protected` | Declaring class, subclasses, same package | Inherited members you want subclasses to reach |
| *(no modifier)* | Same package only | Rare on the exam |

- Fields `private`, methods `public` - that is encapsulation, and it is worth points.
- A `private` field is invisible even to a subclass. Reach it through a getter or a public method.
- A `private` method cannot be called from the driver, even with an object.
- Constructors can be `private` (a utility class that should never be instantiated).

```
public class Student {
    private String name;          // hidden from everything outside Student
    public String getName() { return name; }   // controlled read access
}
```

## 2. `static`: One Copy Per Class

| Form | Meaning | Example |
|---|---|---|
| `static` field | One shared copy for the whole class | `private static int count = 0;` |
| `static` method | Runs without an object | `Math.random()`, `main` |
| `static final` field | A constant | `private static final int MAX = 10;` |

Rules that generate exam errors:

- A `static` method **cannot** use `this`, and cannot touch instance fields or call instance methods directly.
- `main` is always `static`; to call an instance method from it you need `new ClassName().method()`.
- Static state is global: modifying it through one object is visible through every other object.
- `obj.staticMethod()` compiles but is bad style; write `ClassName.staticMethod()`.
- Overriding does not apply to `static` methods - a subclass version **hides** the parent's, and the call is resolved at compile time.

```
public class Counter {
    private static int total = 0;   // shared
    private int id;                 // per object

    public Counter() { total++; id = total; }
    public static int getTotal() { return total; }   // no this inside
}
```

## 3. `final`: Assigned Once

| Applied to | Effect | Trap |
|---|---|---|
| Variable / field | Can be assigned exactly once | Second assignment is `CE: cannot assign a value to final variable` |
| Method | Cannot be overridden | Subclass override is a CE |

| Applied to | Effect | Trap |
|---|---|---|
| Class | Cannot be extended | `extends` it and you get a CE |

- Constant convention: `private static final double TAX_RATE = 0.08;` - uppercase, `static` so there is one copy, `final` so nobody reassigns.
- A `final` **instance** field must be assigned in every constructor.
- `final` and `static` are independent: `final` alone means "per object but write-once".
- **`final` does not freeze contents.** `final int[] a` locks the *reference* only; `a[0] = 5;` is perfectly legal. Same for `final ArrayList`.
- `final` does not mean "known at compile time"; it means "assigned once, at the latest inside the constructor".

### 4. `void`: Returns Nothing

- `public void doSomething(...)` performs an action and hands nothing back.
- `return;` is legal inside a `void` method and exits early; `return value;` is a CE.
- You cannot use a `void` call as a value: `int x = reset();` and `System.out.println(reset());` are both CEs.
- `void` is not a type you can declare a variable of.

### 5. `this` and `super`: The Object's Two Pointers

| Keyword | Refers to | Legal where | Illegal where |
|---|---|---|---|
| `this` | The current object | Instance methods, constructors | Any `static` context |
| `this(...)` | Another constructor of this class | First statement of a constructor | Anywhere else |
| `super` | The parent object's members | Instance methods of a subclass | Static context |
| `super(...)` | The parent constructor | First statement of a subclass constructor | Anywhere else |
| `super.method()` | The parent's version of a method | Subclass instance methods | Static context |

- Shadowing fix: `this.name = name;` - left side the field, right side the parameter.
- `super(...)` must be the first statement. If you omit it, the compiler inserts a no-argument `super()`; if the parent has no no-arg constructor, you get a CE.
- You cannot use both `this(...)` and `super(...)` in the same constructor.
- `super.toString()` is the standard way to reuse the parent's formatting in an override.
- `this` inside `main` is a CE, because `main` is static.

```
public SavingsAccount(String owner, double balance, double rate) {
    super(owner, balance);   // first statement, no exceptions
    this.rate = rate;        // this = the object being built
}
```

### 6. `new`: Create the Object

| Form | Creates | Note |
|---|---|---|
| `new ClassName(args)` | One object | Runs the matching constructor |
| `new int[5]` | Array of 5 zeros | Does **not** run any constructor |
| `new int[] {1, 2, 3}` | Array with values | Size comes from the list |
| `new ArrayList<String>()` | Empty list | Declaring the variable does not create the list |

- A declared reference with no `new` is `null`; the first method call on it throws `NullPointerException`.
- `new` on an interface or abstract class is a CE.
- Write the element type on the right as the exam materials do: `new ArrayList<String>()`.
- Creating an array of objects gives you an array of `null` slots - each element still needs its own `new`.

### 7. `null`: The Absence of an Object

- Default value of object fields and array-of-object slots. **Not** the default of local variables (those must be assigned).
- `null` is not `0` and not `""`. `s.length()` on a `null` String throws `NullPointerException`.
- Test with `== null` / `!= null`; never `.equals(null)` on the variable itself - but `"word".equals(s)` is safe even when `s` is null.
- A `null` passed to a method compiles fine and blows up at runtime.
- Short-circuit `&&` is the standard guard: `if (s != null && s.length() > 0)`.

### 8. `extends`: Inheritance

- A subclass inherits `public` and `protected` members; it does **not** inherit constructors or `private` members.
- Java allows exactly one superclass per class.
- A subclass constructor must reach a superclass constructor: explicitly with `super(...)`, or implicitly with a no-arg `super()` that must exist.
- An overridden method must keep the same signature. Same name but different parameter types is an **overload**, which quietly leaves the parent method in charge.

```
class Animal { public String speak() { return "..."; } }
class Dog extends Animal {
    @Override public String speak() { return "Woof"; }   // override: same signature
    public String speak(int times) { return "Woof"; }     // overload: different list
}
```

### 9. `return`: Exit and Hand Back

- Non-`void`: `return expression;` - the expression must be assignable to the declared return type.
- `void`: `return;` only.
- Everything after a `return` in the same block is unreachable -> `CE: unreachable statement`.
- A `return` inside a loop ends the method, not just the iteration (that is `break`'s job).

### 10. Control-Flow Keywords That Show Up

| Keyword | Does | Notes |
|---|---|---|
| `break` | Exits the innermost loop or `switch` | Does not exit nested loops, only the one it sits in |
| `switch` / `case` / `default` | Multi-way branch on `int`, `char`, `String` | Missing `break` = fall-through into the next case |
| `while` / `for` / `do` | Loops | `do` runs the body at least once |
| `if` / `else` | Branch | `else if` chains are checked top-down; the first true branch wins |
| `instanceof` | Type test before a cast | Usually paired with a downcast |

> `continue` exists in Java and skips to the next iteration, but it is not part of the printed AP CSA subset - recognize it, do not rely on it.

### 11. Combinations You Type Constantly

| Declaration | Why each word is there |
|---|---|
| `public static void main(String[] args)` | `public` so the JVM can call it, `static` so no object is needed, `void` because nothing is returned |
| `private static final int LIMIT = 10;` | `private` encapsulated, `static` one copy, `final` write-once |
| `public String toString()` | `public` so `println` and string concatenation can call it, returns `String` |
| `private int count;` | Encapsulated per-object state |
| `public Student(String n)` | Constructor: `public`, no return type at all |

| Declaration | Why each word is there |
|---|---|
| `public boolean withdraw(double amt)` | Returns success/failure instead of printing |
| `public static double toCelsius(double f)` | Pure utility: no object state needed |

## 12. Words You Cannot Use as Identifiers

`abstract`, `boolean`, `break`, `case`, `catch`, `char`, `class`, `const`, `continue`, `default`, `do`, `double`, `else`, `extends`, `false`, `final`, `finally`, `for`, `if`, `implements`, `import`, `int`, `interface`, `long`, `new`, `null`, `package`, `private`, `protected`, `public`, `return`, `short`, `static`, `super`, `switch`, `this`, `throw`, `throws`, `true`, `try`, `void`, `while`

Naming a variable `class` or `new` produces a confusing cascade of errors; the fix is always a rename.

## 13. Keyword Trap Table

| Written | Error | Correct form |
|---|---|---|
| `this` inside a `static` method | `CE: non-static variable this cannot be referenced` | Remove `this`, or make the method non-static |
| `static` method touching an instance field | `CE: non-static variable ... cannot be referenced` | Pass the object in, or make the field static |
| Reassigning a `final` field | `CE: cannot assign a value to final variable` | Drop `final`, or assign once |
| `return value;` in a `void` method | `CE: incompatible types` | `return;` |
| `void` call used in an expression | `CE: 'void' type not allowed here` | Split the call and the print |
| `super(...)` not first | `CE: call to super must be first statement` | Move it to the top |
| `this(...)` and `super(...)` together | `CE` | Pick one |
| Override with a different parameter list | No error - the parent method still runs | Fix the signature |
| `.equals()` on a possibly-null reference | `NullPointerException` | `constant.equals(var)` or guard with `!= null` |
| `private` field read from a subclass | `CE: ... has private access` | Public getter or `super` accessor |
| Missing `break` in `switch` | Falls through, wrong output | Add `break` |
| `final` assumed to protect array contents | Contents still mutable | Copy the array if it must be sealed |